

These are exactly the right questions to ask **before** we start writing code. The architecture you choose now will affect every feature you build over the next few months.

1. Why `spring.application.name=training-backend`?

It can absolutely be:

```
properties
```

```
spring.application.name=trainingapp-backend
```

or even

```
properties
```

```
spring.application.name=training-app-backend
```

This property is mostly used for:

- logging
- Spring Actuator
- service discovery (if you ever use microservices)

It doesn't affect your application logic.

Since your repository is called **training-app**, I'd use

```
properties
```

```
spring.application.name=trainingapp-backend
```

2. How should we organize Flyway migrations?

The important rule is:

| **One migration = one logical change.**

Not:

| One migration = one table.

If you need to change the **Movement** feature in six months, you'll know exactly where to look.

I would create this structure:

```
src
├── main
│   └── resources
│       ├── db
│       │   └── migration
│       │       ├── V1__create_enum_types.sql
│       │       ├── V2__create_users_and_workspaces.sql
│       │       ├── V3__create_movement_library.sql
│       │       ├── V4__create_training_structure.sql
│       │       ├── V5__create_attempts.sql
│       │       └── V6__create_chat.sql
```

V1

Contains only:

- relationship_type
- media_type
- phase_status
- session_status
- etc.

V2

Contains

```
USER

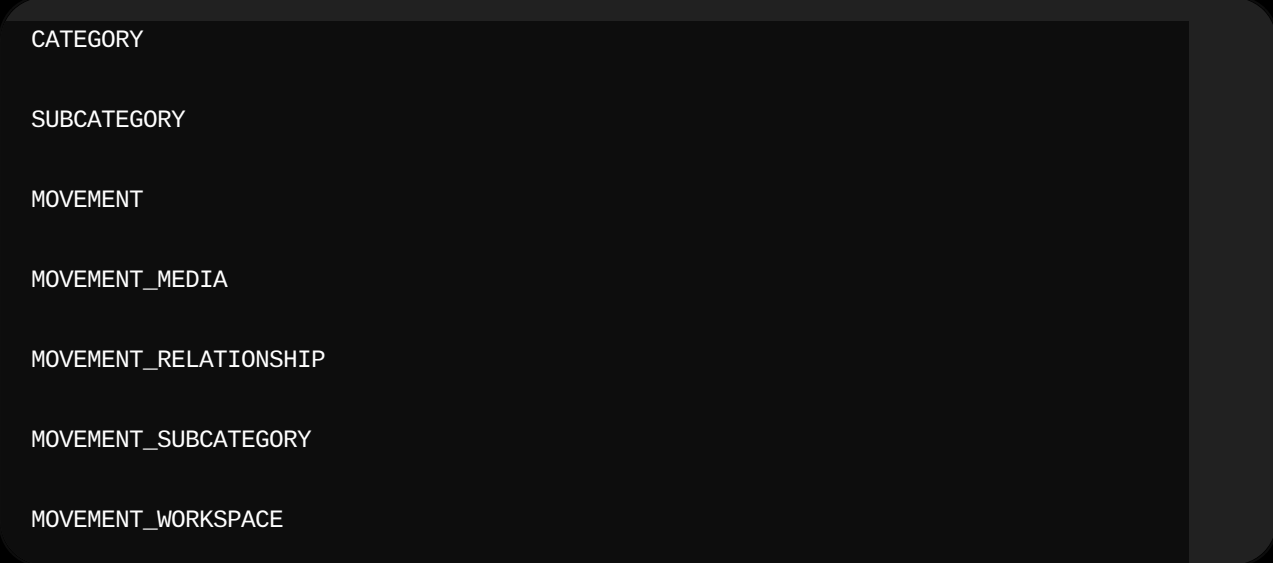
WORKSPACE

USER_WORKSPACE
```

because they depend on each other.

V3

Contains



CATEGORY

SUBCATEGORY

MOVEMENT

MOVEMENT_MEDIA

MOVEMENT_RELATIONSHIP

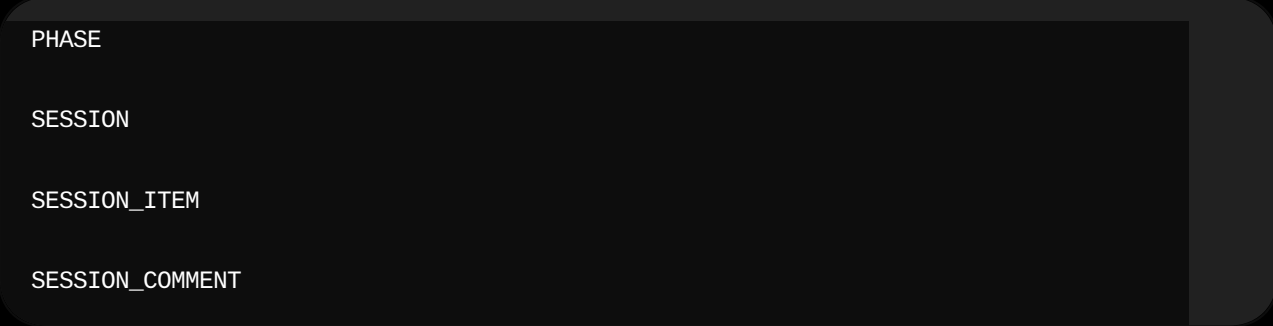
MOVEMENT_SUBCATEGORY

MOVEMENT_WORKSPACE

This is your "Movement Library" feature.

V4

Contains



PHASE

SESSION

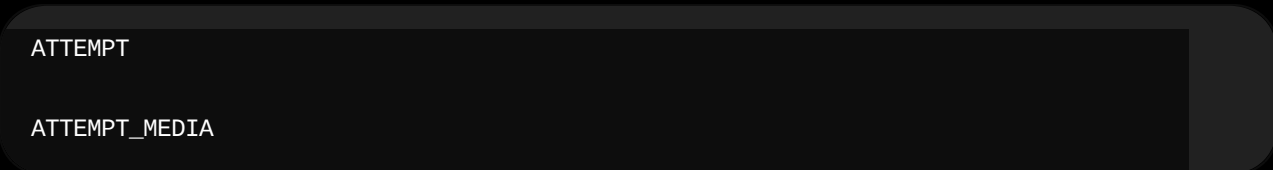
SESSION_ITEM

SESSION_COMMENT

Everything related to programming training.

V5

Contains



ATTEMPT

ATTEMPT_MEDIA

V6

Contains

CHAT

MESSAGE

I think this is much cleaner than having 20 tiny migration files or one giant migration.

3. How should we create entities?

Exactly in the same order.

User

Workspace

UserWorkspace



Category

Subcategory

Movement

MovementRelationship

MovementMedia



Phase

Session

SessionItem

SessionComment



Attempt

AttemptMedia



Chat

Message

Never create all entities at once.

We'll create one feature completely, test it, then move on.

4. How should packages be organized?

This is probably the biggest architectural decision we'll make.

There are **two common styles**.

Option A — Layer-based architecture

This is what is usually taught first.

```
controller
```

```
service
```

```
repository
```

```
entity
```

```
dto
```

```
mapper
```

```
config
```

Inside `controller`:

```
UserController
```

```
MovementController
```

```
ChatController
```

```
PhaseController
```

Inside `service`:

```
UserService
```

```
MovementService
```

```
ChatService
```

etc.

Advantages

Very easy to understand.

Problems

Imagine the app after two years.

```
controller
  38 classes

service
  44 classes

repository
  31 classes

entity
  29 classes

dto
  120 classes
```

Finding everything related to `Movement` means jumping through five different folders.

Option B — Feature-based architecture

Instead:

```
user

movement

phase

chat
```

Inside `movement`

```
MovementController

MovementService

MovementRepository
```

Movement

MovementDto

MovementMapper

Everything is together.

Suppose tomorrow you want to work only on movement uploads.

You open:

movement

Everything is there.

No folder hopping.

This is the architecture I would choose for your project.

5. Then what is **common**?

Good question.

Imagine six months from now.

You'll probably have:

MovementException

ChatException

PhaseException

WorkspaceException

Do they belong inside every feature?

Not really.

Instead:

```
common
    exception
```

contains infrastructure exceptions shared by the application.

Another example.

Every table has

```
created_at
updated_at
```

Instead of repeating them 15 times:

```
@Entity
public class User {

    createdAt

    updatedAt

}
```

```
@Entity
public class Movement {

    createdAt

    updatedAt

}
```

...

We create one class:

```
BaseEntity
```

```
@MappedSuperclass
public abstract class BaseEntity {

    createdAt
```

```
updatedAt
```

```
}
```

Then

```
Movement extends BaseEntity
```

```
User extends BaseEntity
```

```
Chat extends BaseEntity
```

```
Attempt extends BaseEntity
```

No duplicated code.

That's why I'd put it under

```
common/entity
```

Another example.

Soon we'll validate DTOs.

We'll use annotations like

```
Java
```

```
@NotBlank
```

```
@Email
```

```
@Valid
```

Eventually we may create our own.

For example

```
Java
```

```
@ValidPassword
```

or

```
Java
```

@StrongPassword

Those belong to

```
common/validation
```

because every feature might use them.

Utilities.

Suppose later we create

```
UuidUtils
```

```
DateUtils
```

```
FileUtils
```

They're not related to users or movements.

So they go into

```
common/util
```

6. I would slightly improve the `common` package

Instead of

```
common
```

```
    entity
```

```
    exception
```

```
    mapper
```

```
    validation
```

```
    util
```

I'd use

```
common
```

```
    entity
```

```
        BaseEntity.java
```

```
        AuditableEntity.java
```

```
    exception
```

```
        ResourceNotFoundException.java
```

```
        BusinessException.java
```

```
        GlobalExceptionHandler.java
```

```
    mapper
```

```
    validation
```

```
    util
```

```
    dto
```

Notice the addition of

```
dto
```

Sometimes you'll have DTOs shared across multiple features (pagination, API responses, error responses). Keeping them in `common` avoids duplication.

7. The package structure I recommend

For your project, which is expected to grow into a commercial product with many business features, I would use this structure:

```
com.trainingapp
```

```
|
```

```
|— common
```

```
| |— dto
```

```
| |— entity
```

```
| |— exception
```

```
| |— mapper
```

```
| |— util
```

```
| |— validation
```

```
|
```

```
|— config
```

```
|
```

```
|— security
```

```
|
```

```
|— user
```

```
| |— controller
```

```
|   ├── dto
|   ├── entity
|   ├── mapper
|   ├── repository
|   └── service
|
|── workspace
|   ├── controller
|   ├── dto
|   ├── entity
|   ├── mapper
|   ├── repository
|   └── service
|
|── movement
|   ├── controller
|   ├── dto
|   ├── entity
|   ├── mapper
|   ├── repository
|   └── service
|
|── phase
|── session
|── attempt
└── chat
```

This gives you the best of both worlds: the code is organized by business feature, while each feature still has a familiar Spring structure (`controller`, `service`, `repository`, etc.). As the application grows, adding a new domain like `notification` or `billing` simply means adding another top-level feature package without cluttering shared folders. Given your goal of building a commercial coaching platform rather than a small demo, I think this organization will scale much better than a single global `controller`, `service`, and `repository` layout.